

methods, which are much more efficient if there are sharp features in K or f . Variable stepsize methods are quite a bit more complicated than their counterparts for differential equations; we refer you to the literature [1,2] for a discussion.

You should also be on the lookout for singularities in the integrand. If you find them, look to §19.3 for additional ideas.

CITED REFERENCES AND FURTHER READING:

- Linz, P. 1985, *Analytical and Numerical Methods for Volterra Equations* (Philadelphia: S.I.A.M.).[1]
 Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[2]

19.3 Integral Equations with Singular Kernels

Many integral equations have singularities in either the kernel or the solution or both. A simple quadrature method will show poor convergence with N if such singularities are ignored. There is sometimes art in how singularities are best handled.

We start with a few straightforward suggestions:

1. Integrable singularities can often be removed by a change of variable. For example, the singular behavior $K(t, s) \sim s^{1/2}$ or $s^{-1/2}$ near $s = 0$ can be removed by the transformation $z = s^{1/2}$. Note that we are assuming that the singular behavior is confined to K , whereas the quadrature actually involves the product $K(t, s)f(s)$, and it is this product that must be “fixed.” Ideally, you must deduce the singular nature of the product before you try a numerical solution, and take the appropriate action. Commonly, however, a singular kernel does *not* produce a singular solution $f(t)$. (The highly singular kernel $K(t, s) = \delta(t - s)$ is simply the identity operator, for example.)

2. If $K(t, s)$ can be factored as $w(s)\bar{K}(t, s)$, where $w(s)$ is singular and $\bar{K}(t, s)$ is smooth, then a Gaussian quadrature based on $w(s)$ as a weight function will work well. Even if the factorization is only approximate, the convergence is often improved dramatically. All you have to do is replace `gauleg` in the routine `fred2` by another quadrature routine. Section 4.6 explained how to construct such quadratures; or you can find tabulated abscissas and weights in the standard references [1,2]. You must of course supply $\bar{K}$ instead of K .

This method is a special case of the *product Nystrom method* [3,4], where one factors out a singular term $p(t, s)$ depending on both t and s from K and constructs suitable weights for its Gaussian quadrature. The calculations in the general case are quite cumbersome, because the weights depend on the chosen $\{t_i\}$ as well as the form of $p(t, s)$.

We prefer to implement the product Nystrom method on a uniform grid, with a quadrature scheme that generalizes the extended Simpson’s 3/8 rule (equation 4.1.5) to arbitrary weight functions. We discuss this in the subsections below.

3. Special quadrature formulas are also useful when the kernel is not strictly singular, but is “almost” so. One example is when the kernel is concentrated near $t = s$ on a scale much smaller than the scale on which the solution $f(t)$ varies. In that case, a quadrature formula can be based on locally approximating $f(s)$ by a polynomial or spline, while calculating the first few *moments* of the kernel $K(t, s)$ at the tabulation points t_i . In such a scheme the narrow width of the kernel becomes an asset, rather than a liability: The quadrature becomes exact as the width of the kernel goes to zero.

4. An infinite range of integration is also a form of singularity. Truncating the range at a large finite value should be used only as a last resort. If the kernel goes rapidly to zero, then a Gauss-Laguerre [$w \sim \exp(-\alpha s)$] or Gauss-Hermite [$w \sim \exp(-s^2)$] quadrature should work well. Long-tailed functions often succumb to the transformation

$$s = \frac{2\alpha}{z + 1} - \alpha \quad (19.3.1)$$

which maps $0 < s < \infty$ to $1 > z > -1$ so that Gauss-Legendre integration can be used. Here $\alpha > 0$ is a constant that you adjust to improve the convergence.

5. A common situation in practice is that $K(t, s)$ is singular along the diagonal line $t = s$. Here the Nystrom method fails completely because the kernel gets evaluated at (t_i, s_i) . *Subtraction of the singularity* is one possible cure:

$$\begin{aligned} \int_a^b K(t, s) f(s) ds &= \int_a^b K(t, s) [f(s) - f(t)] ds + \int_a^b K(t, s) f(t) ds \\ &= \int_a^b K(t, s) [f(s) - f(t)] ds + r(t) f(t) \end{aligned} \quad (19.3.2)$$

where $r(t) = \int_a^b K(t, s) ds$ is computed analytically or numerically. If the first term on the right-hand side is now regular, we can use the Nystrom method. Instead of equation (19.1.4), we get

$$f_i = \lambda \sum_{\substack{j=0 \\ j \neq i}}^{N-1} w_j K_{ij} [f_j - f_i] + \lambda r_i f_i + g_i \quad (19.3.3)$$

Sometimes the subtraction process must be repeated before the kernel is completely regularized. See [3] for details. (And read on for a different, we think better, way to handle diagonal singularities.)

19.3.1 Quadrature on a Uniform Mesh with Arbitrary Weight

It is possible in general to find n -point linear quadrature rules that approximate the integral of a function $f(x)$, times an arbitrary weight function $w(x)$, over an arbitrary range of integration (a, b) , as the sum of weights times n evenly spaced values of the function $f(x)$, say at $x = kh, (k+1)h, \dots, (k+n-1)h$. The general scheme for deriving such quadrature rules is to write down the n linear equations that must be satisfied if the quadrature rule is to be exact for the n functions $f(x) = \text{const}, x, x^2, \dots, x^{n-1}$, and then solve these for the coefficients. This can be done analytically, once and for all, if the moments of the weight function over the same range of integration,

$$W_n \equiv \frac{1}{h^n} \int_a^b x^n w(x) dx \quad (19.3.4)$$

are assumed to be known. Here the prefactor h^{-n} is chosen to make W_n scale as h if (as in the usual case) $b - a$ is proportional to h .

Carrying out this prescription for the four-point case gives the result

$$\begin{aligned} \int_a^b w(x) f(x) dx &= \\ &\frac{1}{6} f(kh) [(k+1)(k+2)(k+3)W_0 - (3k^2 + 12k + 11)W_1 + 3(k+2)W_2 - W_3] \\ &+ \frac{1}{2} f([k+1]h) [-k(k+2)(k+3)W_0 + (3k^2 + 10k + 6)W_1 - (3k+5)W_2 + W_3] \\ &+ \frac{1}{2} f([k+2]h) [k(k+1)(k+3)W_0 - (3k^2 + 8k + 3)W_1 + (3k+4)W_2 - W_3] \\ &+ \frac{1}{6} f([k+3]h) [-k(k+1)(k+2)W_0 + (3k^2 + 6k + 2)W_1 - 3(k+1)W_2 + W_3] \end{aligned} \quad (19.3.5)$$

While the terms in brackets superficially appear to scale as k^2 , there is typically cancellation at both $O(k^2)$ and $O(k)$.

Equation (19.3.5) can be specialized to various choices of (a, b) . The obvious choice is $a = kh, b = (k+3)h$, in which case we get a four-point quadrature rule that generalizes

Simpson's 3/8 rule (equation 4.1.5). In fact, we can recover this special case by setting $w(x) = 1$, in which case (19.3.4) becomes

$$W_n = \frac{h}{n+1}[(k+3)^{n+1} - k^{n+1}] \quad (19.3.6)$$

The four terms in square brackets in equation (19.3.5) each become independent of k , and (19.3.5) in fact reduces to

$$\int_{kh}^{(k+3)h} f(x)dx = \frac{3h}{8}f(kh) + \frac{9h}{8}f([k+1]h) + \frac{9h}{8}f([k+2]h) + \frac{3h}{8}f([k+3]h) \quad (19.3.7)$$

Back to the case of general $w(x)$, some other choices for a and b are also useful. For example, we may want to choose (a, b) to be $([k+1]h, [k+3]h)$ or $([k+2]h, [k+3]h)$, allowing us to finish off an extended rule whose number of intervals is not a multiple of three, without loss of accuracy: The integral will be estimated using the four values $f(kh), \dots, f([k+3]h)$. Even more useful is to choose (a, b) to be $([k+1]h, [k+2]h)$, thus using four points to integrate a centered single interval. These weights, when sewed together into an extended formula, give quadrature schemes that have smooth coefficients, i.e., without the Simpson-like 2, 4, 2, 4, 2 alternation. (In fact, this was the technique that we used to derive equation 4.1.14, which you may now wish to re-examine.)

All these rules are of the same order as the extended Simpson's rule, that is, exact for $f(x)$ a cubic polynomial. Rules of lower order, if desired, are similarly obtained. The three-point formula is

$$\begin{aligned} \int_a^b w(x)f(x)dx &= \frac{1}{2}f(kh)[(k+1)(k+2)W_0 - (2k+3)W_1 + W_2] \\ &\quad + f([k+1]h)[-k(k+2)W_0 + 2(k+1)W_1 - W_2] \\ &\quad + \frac{1}{2}f([k+2]h)[k(k+1)W_0 - (2k+1)W_1 + W_2] \end{aligned} \quad (19.3.8)$$

Here the simple special case is to take $w(x) = 1$, so that

$$W_n = \frac{h}{n+1}[(k+2)^{n+1} - k^{n+1}] \quad (19.3.9)$$

Then equation (19.3.8) becomes Simpson's rule,

$$\int_{kh}^{(k+2)h} f(x)dx = \frac{h}{3}f(kh) + \frac{4h}{3}f([k+1]h) + \frac{h}{3}f([k+2]h) \quad (19.3.10)$$

For nonconstant weight functions $w(x)$, however, equation (19.3.8) gives rules of one order less than Simpson, since they do not benefit from the extra symmetry of the constant case.

The two-point formula is simply

$$\int_{kh}^{(k+1)h} w(x)f(x)dx = f(kh)[(k+1)W_0 - W_1] + f([k+1]h)[-kW_0 + W_1] \quad (19.3.11)$$

Here is a routine `Wwghts` that uses the above formulas to compute an extended N -point quadrature rule for the interval $(a, b) = (0, [N-1]h)$. Input to `Wwghts` is a user-supplied object called `quad` in the routine. This object must contain a function `kermom`, which is called to get the first four *indefinite-integral* moments of $w(x)$, namely

$$F_m(y) \equiv \int^y s^m w(s)ds \quad m = 0, 1, 2, 3 \quad (19.3.12)$$

(The lower limit is arbitrary and can be chosen for convenience.) Cautionary note: When called with $N < 4$, `Wwghts` returns a rule of lower order than Simpson; you should structure your problem to avoid this.

fred_singular.h

```

template <class Q>
struct Wwghts {
Constructs weights for the n-point equal-interval quadrature from 0 to (n-1)h of a function  $f(x)$ 
times an arbitrary (possibly singular) weight function  $w(x)$ . The indefinite-integral moments
 $F_n(y)$  of  $w(x)$  are provided by the user-supplied function kermom in the quad object.
    Doub h;
    Int n;
    Q &quad;
    VecDoub wghts;
    Wwghts(Doub hh, Int nn, Q &q) : h(hh), n(nn), quad(q), wghts(n) {}
    Constructor arguments are h, n, and the user-supplied quad object.
    VecDoub weights()
    This function returns the weights in wghts[0..n-1].
    {
        Int k;
        Doub fac;
        Doub hi=1.0/h;
        for (Int j=0;j<n;j++)
            wghts[j]=0.0;
        if (n >= 4) {
            VecDoub wold(4),wnew(4),w(4);
            wold=quad.kermom(0.0);
            Doub b=0.0;
            for (Int j=0;j<n-3;j++) {
                Doub c=j;
                Doub a=b;
                b=a+h;
                if (j == n-4) b=(n-1)*h;
                wnew=quad.kermom(b);
                for (fac=1.0,k=0;k<4;k++,fac*=hi)
                    w[k]=(wnew[k]-wold[k])*fac;
                wghts[j] += (((c+1.0)*(c+2.0)*(c+3.0)*w[0] Equation (19.3.5).
                    -(11.0+c*(12.0+c*3.0))*w[1]+3.0*(c+2.0)*w[2]-w[3])/6.0);
                wghts[j+1] += ((-c*(c+2.0)*(c+3.0)*w[0]
                    +(6.0+c*(10.0+c*3.0))*w[1]-(3.0*c+5.0)*w[2]+w[3])*0.5);
                wghts[j+2] += ((c*(c+1.0)*(c+3.0)*w[0]
                    -(3.0+c*(8.0+c*3.0))*w[1]+(3.0*c+4.0)*w[2]-w[3])*0.5);
                wghts[j+3] += ((-c*(c+1.0)*(c+2.0)*w[0]
                    +(2.0+c*(6.0+c*3.0))*w[1]-3.0*(c+1.0)*w[2]+w[3])/6.0);
                for (k=0;k<4;k++) wold[k]=wnew[k];
            }
            Reset lower limits for moments.
        } else if (n == 3) {
            Lower-order cases; not recommended.
            VecDoub wold(3),wnew(3),w(3);
            wold=quad.kermom(0.0);
            wnew=quad.kermom(h+h);
            w[0]=wnew[0]-wold[0];
            w[1]=hi*(wnew[1]-wold[1]);
            w[2]=hi*hi*(wnew[2]-wold[2]);
            wghts[0]=w[0]-1.5*w[1]+0.5*w[2];
            wghts[1]=2.0*w[1]-w[2];
            wghts[2]=0.5*(w[2]-w[1]);
        } else if (n == 2) {
            VecDoub wold(2),wnew(2),w(2);
            wold=quad.kermom(0.0);
            wnew=quad.kermom(h);
            wghts[0]=wnew[0]-wold[0]-(wghts[1]=hi*(wnew[1]-wold[1]));
        }
        return wghts;
    }
};

```

We will now give an example of how to apply Wwghts to a singular integral equation.

19.3.2 Worked Example: A Diagonally Singular Kernel

As a particular example, consider the integral equation

$$f(x) + \int_0^\pi K(x, y) f(y) dy = \sin x \quad (19.3.13)$$

with the (arbitrarily chosen) nasty kernel

$$K(x, y) = \cos x \cos y \times \begin{cases} -\ln(x-y) & y < x \\ \sqrt{y-x} & y \geq x \end{cases} \quad (19.3.14)$$

which has a logarithmic singularity on the left of the diagonal, combined with a square-root discontinuity on the right.

The first step is to do (analytically, in this case) the required moment integrals over the singular part of the kernel, equation (19.3.12). Since these integrals are done at a fixed value of x , we can use x as the lower limit. For any specified value of y , the required indefinite integral is then either

$$F_m(y; x) = \int_x^y s^m (s-x)^{1/2} ds = \int_0^{y-x} (x+t)^m t^{1/2} dt \quad \text{if } y > x \quad (19.3.15)$$

or

$$F_m(y; x) = -\int_x^y s^m \ln(x-s) ds = -\int_0^{x-y} (x-t)^m \ln t dt \quad \text{if } y < x \quad (19.3.16)$$

(where a change of variable has been made in the second equality in each case). Doing these integrals analytically (e.g., using a symbolic integration package), we package the resulting formulas in the function `kermom` in the following routine, `Quad_matrix`. Note that `w(j+1)` returns $F_j(y; x)$. The constructor of `Quad_matrix` calls `Wgghts` to get the quadrature weights and then constructs the quadrature matrix.

```
struct Quad_matrix {
```

[fred_singular.h](#)

Constructs in `a[0..n-1][0..n-1]` the quadrature matrix for an example Fredholm equation of the second kind.

```
    Int n;
```

```
    Doub x;
```

Communicates with `kermom`.

```
    Quad_matrix(MatDoub_0 &a) : n(a.nrows())
```

The constructor obtains the quadrature weights that integrate the singular part of the kernel via calls to `Wgghts`. It then sums the weights with the nonsingular part of the kernel to obtain the quadrature matrix.

```
{
```

```
    const Doub PI=3.14159263589793238;
```

```
    VecDoub wt(n);
```

```
    Doub h=PI/(n-1);
```

```
    Wgghts<Quad_matrix> w(h,n,*this);
```

```
    for (Int j=0; j<n; j++) {
```

```
        x=j*h;
```

Set `x` for `kermom`.

```
        wt=w.weights();
```

```
        Doub cx=cos(x);
```

Part of nonsingular kernel.

```
        for (Int k=0; k<n; k++)
```

Put together all the pieces of the kernel.

```
            a[j][k]=wt[k]*cx*cos(k*h);
```

```
        ++a[j][j];
```

For equations of the second kind, there is a diagonal piece independent of `h`.

```
    }
```

```
}
```

```
VecDoub kermom(const Doub y)
```

Returns `w[0..m-1]`, the first `m` indefinite-integral moments of one row of the singular part of the kernel. (For this example, `m` is hard-wired to be 4.) The input variable `y` labels the column, while the member variable `x` is the row. We can take `x` as the lower limit of integration. Thus, we return the moment integrals either purely to the left or purely to the right of the diagonal.

```
{
```

```

Doub d,df,clog,x2,x3,x4,y2;
VecDoub w(4);
if (y >= x) {
    d=y-x;
    df=2.0*sqrt(d)*d;
    w[0]=df/3.0;
    w[1]=df*(x/3.0+d/5.0);
    w[2]=df*((x/3.0 + 0.4*d)*x + d*d/7.0);
    w[3]=df*((x/3.0 + 0.6*d)*x + 3.0*d*d/7.0)*x+d*d*d/9.0);
} else {
    x3=(x2=x*x)*x;
    x4=x2*x2;
    y2=y*y;
    d=x-y;
    w[0]=d*((clog=log(d))-1.0);
    w[1] = -0.25*(3.0*x+y-2.0*clog*(x+y))*d;
    w[2]=(-11.0*x3+y*(6.0*x2+y*(3.0*x+2.0*y))
        +6.0*clog*(x3-y*y2))/18.0;
    w[3]=(-25.0*x4+y*(12.0*x3+y*(6.0*x2+y*
        (4.0*x+3.0*y)))+12.0*clog*(x4-(y2*y2)))/48.0;
}
return w;
}
};

```

Finally, we solve the linear system for any particular right-hand side, here $\sin x$.

fred_singular.h

Int main_fredex(void)

This sample program shows how to solve a Fredholm equation of the second kind using the product Nystrom method and a quadrature rule especially constructed for a particular, singular, kernel.

```

{
    const Int N=40;                Here the size of the grid is specified.
    const Doub PI=3.141592653589793238;
    VecDoub g(N);
    MatDoub a(N,N);
    Quad_matrix qmx(a);            Make the quadrature matrix; all the action is here.
    LUdcmp alu(a);                 Decompose the matrix.
    for (Int j=0;j<N;j++)          Construct the right-hand side, here sin x.
        g[j]=sin(j*PI/(N-1));
    alu.solve(g,g);                Backsubstitute.
    for (Int j=0;j<N;j++) {        Write out the solution.
        Doub x=j*PI/(N-1);
        cout << fixed << setprecision(2) << setw(6) << (j+1);
        cout << setprecision(6) << setw(13) << x << setw(13) << g[j] << endl;
    }
    return 0;
}

```

With $N = 40$, this program gives accuracy at about the 10^{-5} level. The accuracy increases as N^4 (as it should for our Simpson-order quadrature scheme) *despite* the highly singular kernel. Figure 19.3.1 shows the solution obtained, also plotting the solution for smaller values of N , which are themselves seen to be remarkably faithful. Notice that the solution is smooth, even though the kernel is singular, a common occurrence.

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nrl.com/aands>. [1]
- Stroud, A.H., and Secrest, D. 1966, *Gaussian Quadrature Formulas* (Englewood Cliffs, NJ: Prentice-Hall). [2]